

Technical University of Cluj-Napoca

Faculty of Automation and Computer Science

Software Design Project 2026

UTCN Extracurricular Events

Client-Server Application

Student: Stefania Mozacu

Group: 30434

Specialization: Computer Engineering, English Program

Academic Year: 2025–2026

Date: March 17, 2026

Contents

1	Introduction	2
2	Project Overview	2
2.1	Main Objectives	2
2.2	User Roles	3
3	Functional Requirements	3
3.1	Student Functionalities	3
3.2	Organizer Functionalities	4
3.3	Admin Functionalities	4
4	Database Design	5
4.1	Main Tables	5
4.2	Entity Description	6
4.3	Relationships	6
4.4	Normalized Database Schema	6
5	Activity Diagrams	7
5.1	Student Registration Workflow	7
5.2	Organizer Activity Diagram	8
5.3	Admin Activity Diagram	9
6	Sequence Diagram	9
6.1	Event Registration Flow	10
7	API Design	10
7.1	Authentication Endpoints	10
7.2	Event Endpoints	10
7.3	Registration Endpoints	10
7.4	Administrative Endpoints	11
8	Chosen Technologies	11
9	Conclusion	11

1. Introduction

UTCN Extracurricular Events is a client-server application designed to support the management of extracurricular activities within the Technical University of Cluj-Napoca. The main idea behind this project comes from a real need that I have observed as the coordinator of Google Developer Group on Campus UTCN. In practice, student events are often promoted separately, through social media pages, group chats, or isolated communication channels, which makes it difficult to ensure that all students see them and stay informed.

I chose this project because I feel the need for a centralized digital space where events can be posted in a structured way and where students have a single place to check for opportunities relevant to them. Such a platform would not only help student organizations communicate more efficiently, but would also make extracurricular activities more visible and accessible.

Another important motivation is coordination between organizations. At UTCN, multiple student organizations such as OSUT, BEST, and others may organize events during the same period. Without a shared overview, these events can overlap in time, reducing participation and creating conflicts in planning. By centralizing all events in one system, organizers can always keep track of what is already scheduled, avoid unnecessary overlaps, and maintain better visibility over the university's extracurricular ecosystem.

Therefore, this project is both a practical software solution and a response to a real organizational problem. The platform will offer dedicated functionalities for students, organizers, and administrators, while also serving as a modern example of a client-server system with authentication, role-based access control, and database management.

2. Project Overview

The proposed system will be implemented as a client-server application. The backend will expose a REST API responsible for authentication, authorization, event management, and registration processing. The client application will provide interfaces tailored to each role.

2.1. Main Objectives

The system aims to:

- provide a centralized platform for extracurricular events at UTCN;
- allow students to search, filter, and register for events;
- allow organizers to create, update, and manage events;
- allow administrators to manage users, categories, departments, and platform content;
- ensure secure access through authentication and role-based authorization.

2.2. User Roles

The application includes the following roles:

- **Student** - can browse events, register, cancel registrations, and view personal registrations.
- **Organizer** - can create, edit, and delete events, and manage participants.
- **Admin** - can manage users, categories, departments, and supervise the entire platform.

3. Functional Requirements

3.1. Student Functionalities

- Register and log into the platform;
- View all available events;
- Search and filter events by date, category, and department;
- View full details of an event;
- Register for an event;
- Cancel an existing registration;
- View personal registration history.

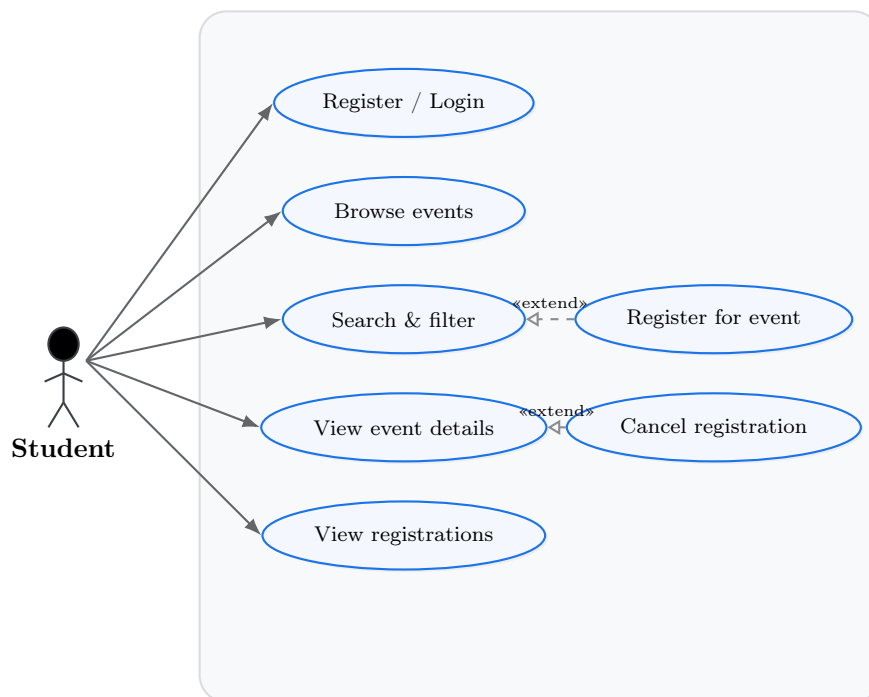


Figure 1: Use-case diagram for the Student actor

3.2. Organizer Functionalities

- Log into the platform;
- Create a new event;
- Update event information;
- Delete an event;
- View the list of registered participants;
- Approve or reject registrations, if approval is required.

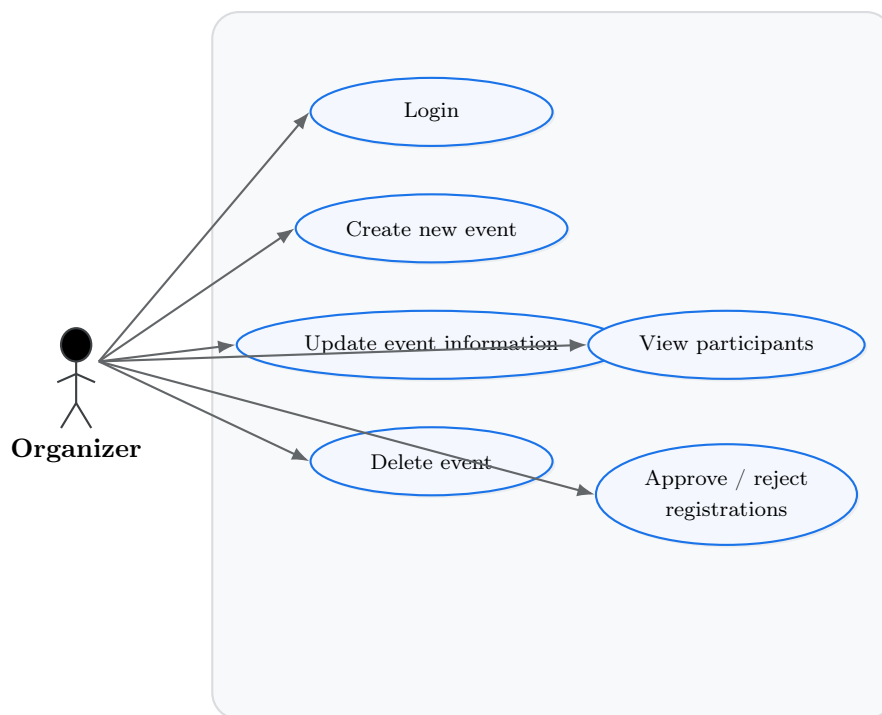


Figure 2: Use-case diagram for the Organizer actor

3.3. Admin Functionalities

- Manage user accounts;
- Approve organizer accounts;
- Manage event categories;
- Manage departments;
- View and moderate all events on the platform.

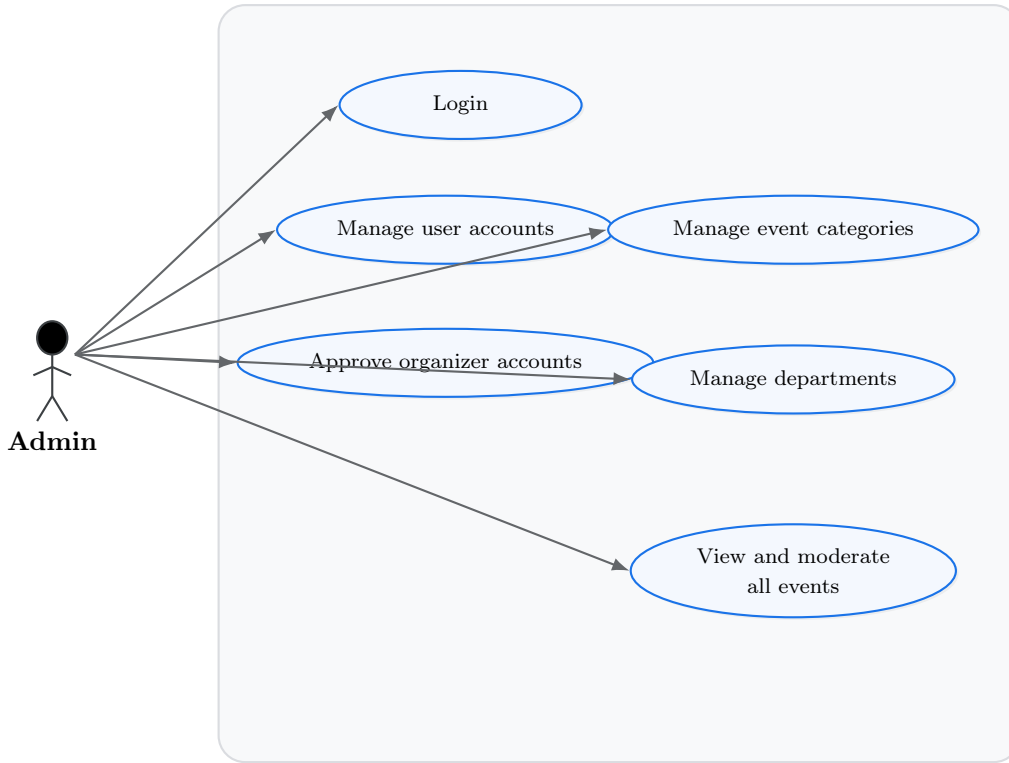


Figure 3: Use-case diagram for the Admin actor

4. Database Design

The database is designed around the core concepts of users, events, registrations, departments, and categories. The model supports the three user roles and the many-to-many relationship between students and events through the `registrations` table.

4.1. Main Tables

Table	Description
<code>users</code>	Stores all platform users, including students, organizers, and administrators.
<code>departments</code>	Stores university departments associated with users and events.
<code>categories</code>	Stores event categories such as workshop, contest, volunteering, or conference.
<code>events</code>	Stores event information such as title, description, location, date, category, department, and organizer.
<code>registrations</code>	Stores student registrations to events and the registration status.

4.2. Entity Description

- **users**(id, name, email, password, role, department_id)
- **departments**(id, name)
- **categories**(id, name)
- **events**(id, title, description, event_date, location, max_participants, organizer_id, category_id, department_id)
- **registrations**(id, student_id, event_id, registration_date, status)

4.3. Relationships

The main relationships in the database are:

- one department has many users;
- one department has many events;
- one category has many events;
- one organizer can create many events;
- one student can have many registrations;
- one event can have many registrations.

Thus, the relationship between students and events is a many-to-many relationship implemented through the **registrations** table.

4.4. Normalized Database Schema

The diagram below illustrates the relational structure, featuring orthogonal relationships and multiplicity indicators.

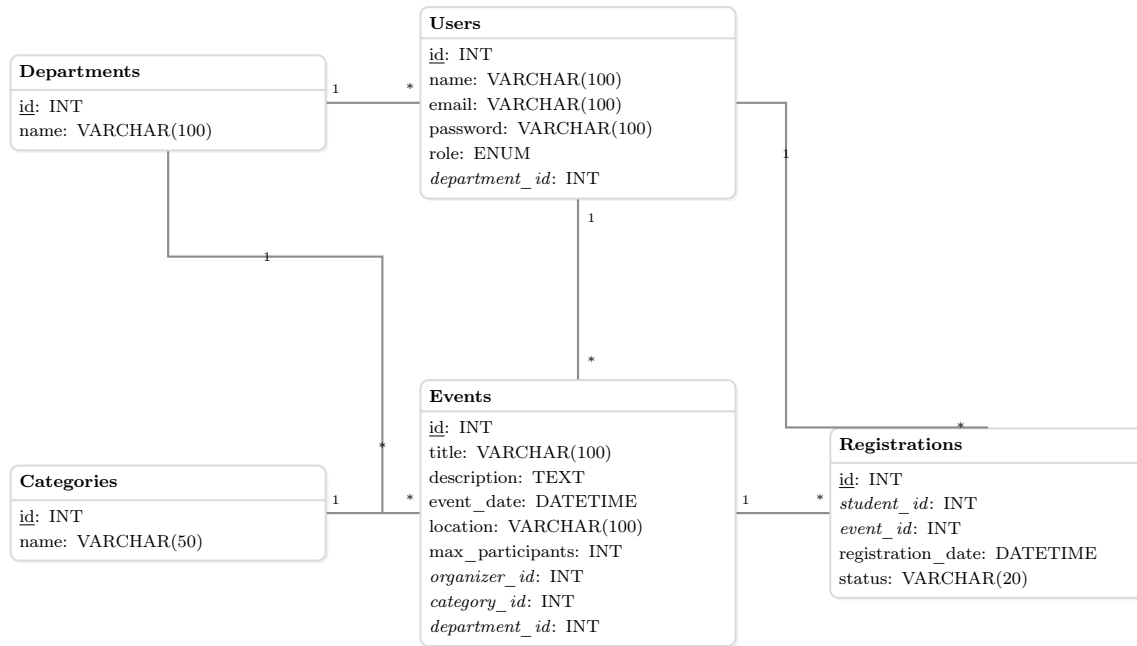


Figure 4: Cleaned ER diagram with orthogonal relationships

5. Activity Diagrams

This section includes one activity diagram for each user role.

5.1. Student Registration Workflow

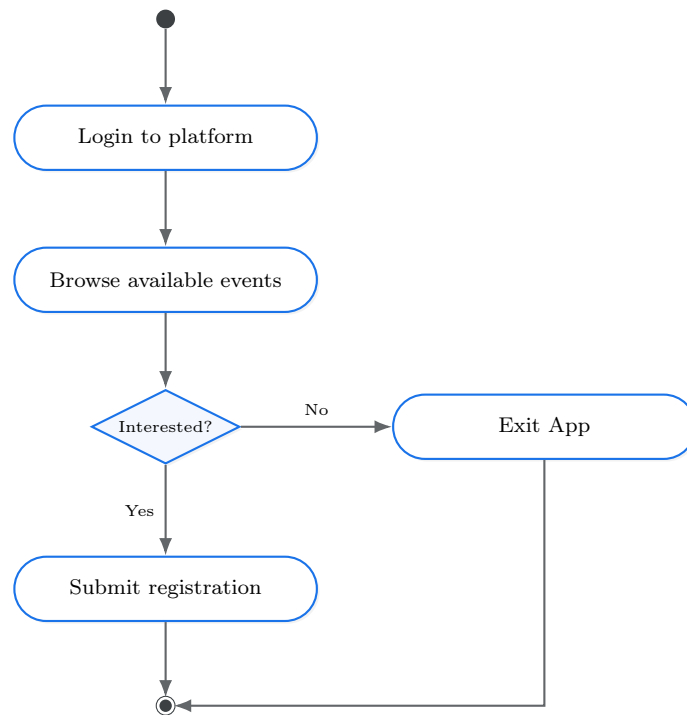


Figure 5: Activity diagram for student registration workflow

5.2. Organizer Activity Diagram

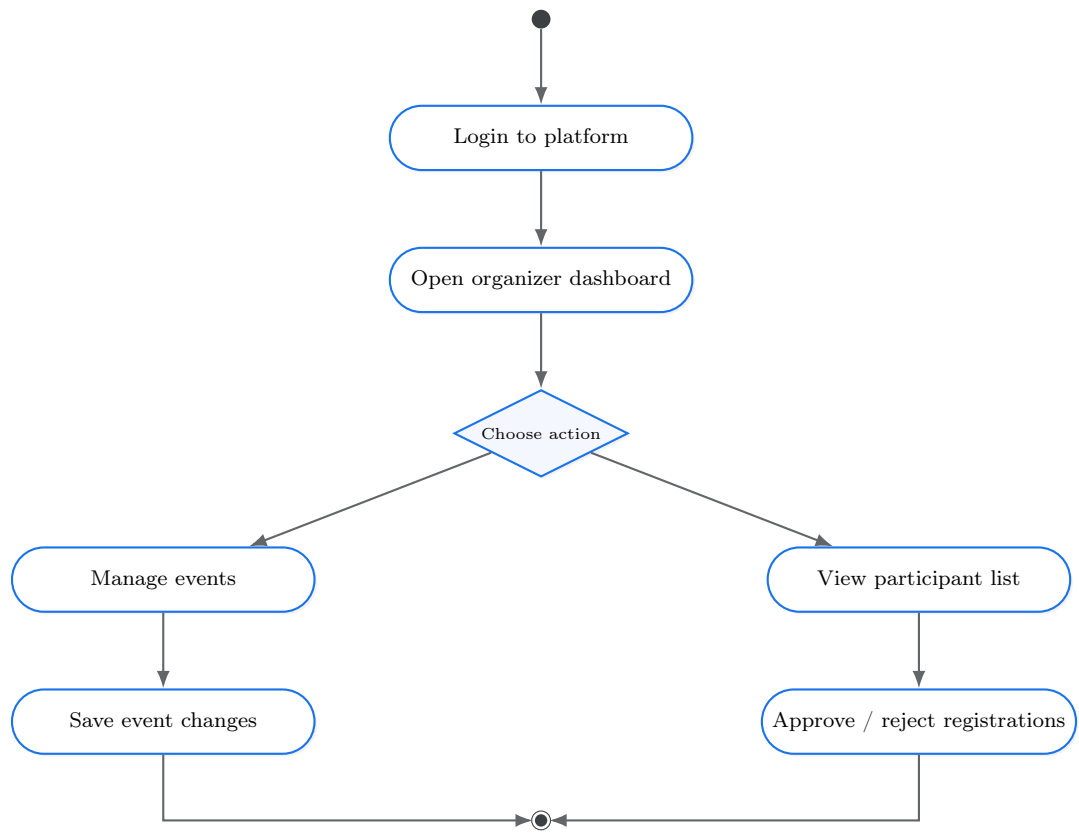


Figure 6: Activity diagram for the Organizer role

5.3. Admin Activity Diagram

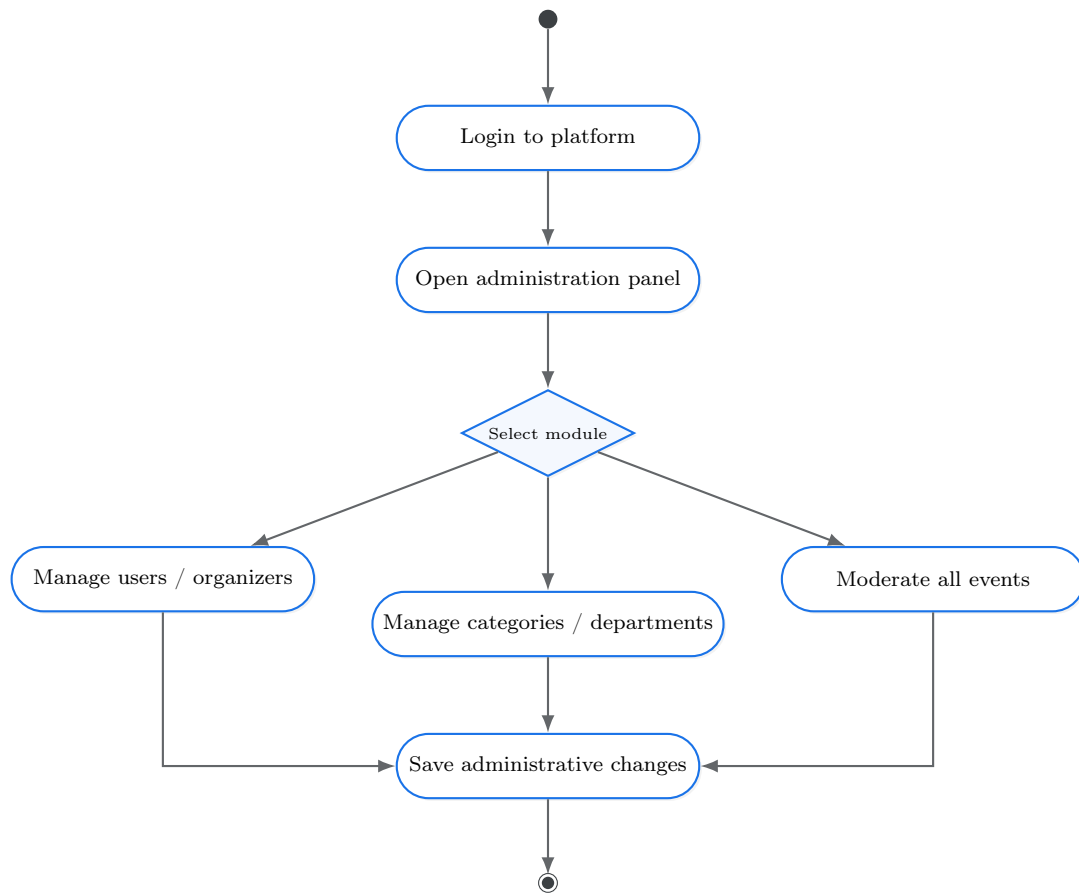


Figure 7: Activity diagram for the Admin role

6. Sequence Diagram

The following sequence diagram illustrates a common interaction flow in the system: a student logs into the platform and then registers for an event. This interaction highlights the communication between the user, the client application, the backend server, and the database.

6.1. Event Registration Flow

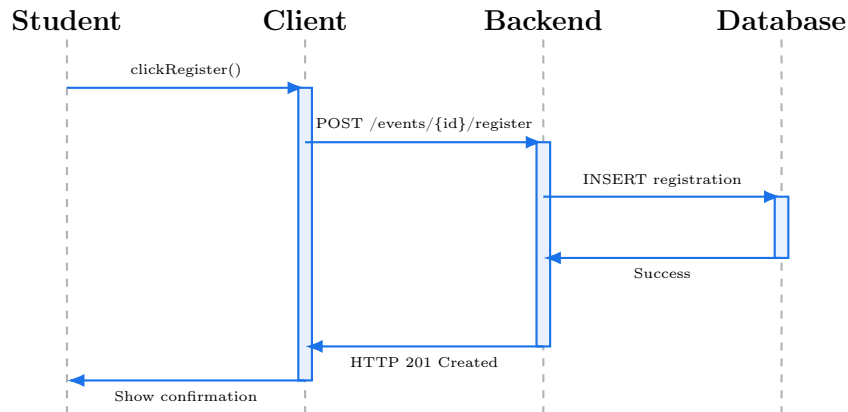


Figure 8: Sequence diagram for event registration flow

7. API Design

The backend of the application will expose a REST API. The API is responsible for authentication, event management, registration handling, and administrative operations. This design allows the system to remain modular and makes it possible to connect multiple clients, such as a web or mobile interface, to the same backend service.

7.1. Authentication Endpoints

- POST /auth/register - creates a new user account;
- POST /auth/login - authenticates a user and returns a token or session.

7.2. Event Endpoints

- GET /events - returns all available events;
- GET /events/{id} - returns the details of a specific event;
- POST /events - creates a new event;
- PUT /events/{id} - updates an existing event;
- DELETE /events/{id} - deletes an event.

7.3. Registration Endpoints

- POST /events/{id}/register - registers a student for an event;
- DELETE /events/{id}/register - cancels a student's registration;
- GET /users/{id}/registrations - returns the registration history of a student.

7.4. Administrative Endpoints

- GET `/users` - returns all platform users;
- PUT `/users/{id}` - updates a user account;
- DELETE `/users/{id}` - removes a user account;
- GET `/categories` - returns all event categories;
- POST `/categories` - creates a new category;
- GET `/departments` - returns all departments;
- POST `/departments` - creates a new department.

8. Chosen Technologies

The proposed implementation stack is the following:

- **Backend:** Kotlin with Spring Boot or Ktor;
- **Frontend:** Web client built with React;
- **Database:** PostgreSQL;
- **Authentication:** JWT-based authentication and role-based authorization.

This stack was chosen because it supports the client-server model very well and allows the implementation of a scalable, maintainable, and modern application.

9. Conclusion

UTCN Extracurricular Events is a practical and relevant client-server application that addresses a real need within the university environment. By centralizing extracurricular opportunities and offering dedicated functionalities for students, organizers, and administrators, the system improves communication, accessibility, and management efficiency.

From a software engineering perspective, the project is suitable for modeling real-world entities and workflows, implementing role-based access control, and designing a relational database with meaningful relationships.